This is a brilliant technical pivot. Moving away from a niche application use case to a **Standardized Benchmark Test Suite** is an elite hackathon strategy. Judges *love* benchmarks because they prove engineering claims with hard, objective data rather than a single flashing demo.

Given the link you provided, **vLLM's Semantic Router** is a spectacular mental model for your project. However, because it is an enterprise-grade infrastructure framework that often relies on Kubernetes, Helm, or proxy configurations, setting it up fully from scratch might swallow up your remaining hours.

The Verdict: Do not build a massive framework from scratch, and do not fight enterprise infrastructure setup. Instead, **build a lightweight, single-script "Simulated Code Harness" in Python or TypeScript** that mimics the *Signal-Driven Decision Routing* core philosophy of vLLM's architecture.

---

1. The Standardized Test: "The Tiered Enterprise Support Suite"

Instead of shopping, your harness will tackle **Customer Support Ticketing & Document Analysis**. It is the perfect playground because requests naturally fall into two distinct buckets: simple, high-frequency tasks, and complex, text-heavy reasoning tasks.

The Dataset (Create a static file with 10 Test Cases)

Create a test_suite.json containing 10 dummy customer tickets:

- **5 "Routine" Tasks (Low Context, Low Reasoning):** e.g., *"How do I change my password?"*, *"What is your return policy window?"*, or *"Can I update my billing email?"*
- **5 "Complex" Tasks (High Context, High Reasoning):** e.g., A ticket containing a massive 2,000-word system error log dump asking: *"My server crashed with this log attached. Is this a memory leak or a database deadlock, and what are the 3 steps to fix it?"*

The Evaluation Metrics

When you run the test suite, your script will evaluate both methods side-by-side using:

1. **Accuracy (Pass/Fail):** Does the model correctly answer the query?
2. **Token Efficiency:** Total Input Tokens vs. Output Tokens.
3. **Financial Cost ($):** Calculated using real API dollar-per-million-token metrics.

---

2. The Model Lineup for Comparison

To make your data compelling, you will compare two setups:

Approach	Routing Strategy	Small/Execution Model	Premium/Reasoning Model
Approach 1: Naive (Status Quo)	None (Send 100% of tasks here)	None	**Claude 3.5 Sonnet** (or GPT-4o)
Approach 2: Multi-Model Harness	Semantic Rule / Fast Classifier	**Llama 3.1 8B** (or Claude 3.5 Haiku)	**Claude 3.5 Sonnet** (The heavy lifter)

---

3. Team Division of Labor (3-Person Plan)

With only a few hours left, you need a "divide and conquer" assembly line. Here is exactly how to split up your team of three:

👤 Person 1: The Dataset & Evaluation Master (Backend Data)

- **Task:** Write the test_suite.json file with the 10 customer service test cases described above.
- **Core Job:** Write the scoring function. This script takes the raw outputs from both approaches, reads the token usage fields (usage.input_tokens and usage.output_tokens) from the API responses, applies standard cost multipliers, and outputs a clean comparison metric file (e.g., results.json).

👤 Person 2: The Harness Architect (Core Logic)

- **Task:** Build the router routing logic function.
- **Core Job:** Create a function routeRequest(ticket). For absolute speed, you can use a quick keyword/length check (e.g., *if ticket length > 500 characters or contains the word 'log', route to Claude Sonnet; otherwise, route to Llama/Haiku*). Once the route is selected, trigger the respective API call and pass the result back to Person 1's evaluator.

👤 Person 3: The UI & Pitch Deck Lead (Frontend & Delivery)

- **Task:** Build a super lightweight visualization page (using Streamlit if Python, or a single Next.js page if TypeScript).
- **Core Job:** Create two progress bars or side-by-side metric boxes showing **Total Cost**

and **Tokens Expended**. While the backend is being wired up, Person 3 should build the presentation slides explaining how your code brings the exact token economic values of the vLLM Semantic Router directly into an application workflow.

The Winning Demo Visual for the Judges

When you present to the judges, your screen should show a clean dashboard layout comparing the real-time execution of the 10 test cases:

Plaintext

```
================================================================================
                    HARNESS BENCHMARK RUN (10/10 TICKETS)
================================================================================

[APPROACH 1: NAIVE COLAUDE 3.5 SONNET]   [APPROACH 2: OUR ROUTING HARNESS]
- Avg Accuracy: 100%                     - Avg Accuracy: 100%
- Total Input Tokens: 25,000             - Premium Tokens: 12,000 (Sonnet)
- Total Output Tokens: 4,000             - Routine Tokens:  13,000 (Llama-8B)
-----------------------------------     -----------------------------------
TOTAL COST: $0.092                       TOTAL COST: $0.038

          🎉 RESULT: 58% COST REDUCTION WITH ZERO LOSS IN ACCURACY! 🎉
```

How does this Customer Support Triage benchmark use case sound to the team? If you like it, let me know if you are writing this in **Python** or **TypeScript**, and I can provide a skeleton structure for the router function to get Person 2 moving immediately!